Interfaces Gráficas y Especialidades en MATLAB

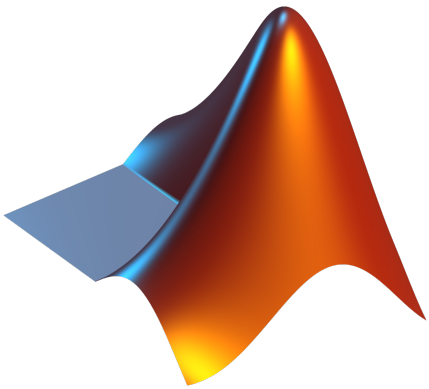
Fabrizio Benavidez[1]

https://github.com/Fabopy/Matlab-curso-

[1] *Facultad de Ciencias Exactas, Físicas y Naturales, Ingeniería Aeronáutica*

Resumen En el siguiente trabajo se “desarrolla" una aplicación hecha en MATLAB para ingresar numerador y denominador y poder seleccionar el tipo de entrada. En la misma aplicación te muestra las gráficas de las entradas que el usuario ingresa. Se utilizó la IA para desarrollar la guía 23 ya que el trabajo está basado en usar *guide*, función que no existe más en MATLAB y pasó a llamarse de otra forma.
Por último, también se vieron algunos trucos o especialidades que tiene MATLAB: generador de números randoms sin que se repita alguno, diferencia entre predefinir el tamaño de un arreglo y cuándo no, tiempos de ejecución del código, definir el tamaño de una figura, cargar imágenes que tiene incorporado MATLAB, algunas funciones como el `why`, `bench` y `life`.

Palabras claves: MATLAB, aplicación, trucos



8 de noviembre de 2026

Semana 11

[1] fabrizio.benavidez@mi.unc.edu.ar

Guía 23: Interfaces Gráficas en MATLAB

En la guía se utiliza el comando *guide* para implementar una interfaz. El tema es que dicho comando quedó en deshuso ya que el sistema migró a *App Designer*[1]. Esto conllevó a usar la inteligencia artificial ya que muchos de los comandos que implementa la guía no existen más. Se utilizó *ChatGPT* para replicar lo mismo que la guía. Se detalla el paso a paso para implementar dicha interfaz o app:

1. **Colocamos los elementos en la interfaz**: *Static Text* se conoce como *Label*; *Edit Text* se conoce como *Edit Field (Text)*; *Pop-up Menu* se conoce como *Drop Down*; *Push Button* se conoce como *Button* y el *Axes* sigue igual.

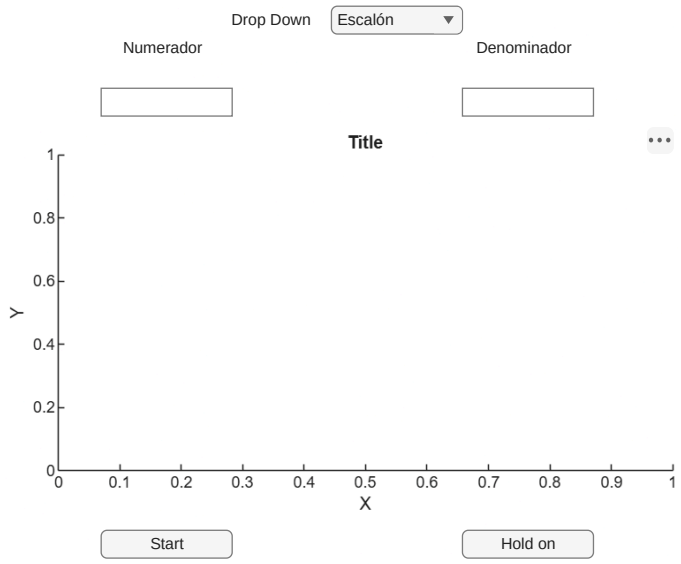


Figura 1. Colocación de los elementos en mi interfaz gráfica

2. **Creación de las propiedades**: En Code View, abrimos Editor → Property → Private Property. App Designer crea un bloque properties (Access = private). Agregamos lo siguiente:

```
    properties (Access = private)
        N = []
        D = []
        v = 'Escalon'
    end

```

`N` es el numerador, `D` es el denominador y `v` es la entrada.

3. **Creamos los callbacks**: En Design View, buscamos app.numEditField en el panel derecho Component Browser. Hacemos clic derecho sobre ese componente y elegimos Callbacks → Add ValueChangedFcn callback. App Designer va a crear la función y nos lleverá a su parte editable. Hacemos lo mismo con el denominador y el desplegable (Dropdown):

```
    % Value changed function: numEditField
        function numEditFieldValueChanged(app, event)
```

[1] https://la.mathworks.com/products/matlab/app-designer.html

```
            app.N = str2num(app.numEditField.Value, Evaluation="
restricted");
            disp(app.N)
        end

        % Value changed function: denEditField
        function denEditFieldValueChanged(app, event)
            app.D = str2num(app.denEditField.Value, Evaluation="
restricted");
            disp(app.D)
        end

        % Value changed function: DropDown
        function DropDownValueChanged(app, event)
            app.v = app.DropDown.Value;

        end

```

4. **Superponer las curvas**: Clic derecho sobre app.HoldonButton → Callbacks → Add ButtonPushedFcn callback. Pegamos este código dentro de la función generada. Hacemos lo mismo con el botón de Start:

```
     % Button pushed function: HoldonButton
        function HoldonButtonPushed(app, event)
            if ishold(app.UIAxes)
                hold(app.UIAxes, 'off');
                app.HoldonButton.Text = 'Hold on';
            else
                hold(app.UIAxes, 'on');
                app.HoldonButton.Text = 'Hold off';
            end
        end

        % Button pushed function: StartButton
        function StartButtonPushed(app, event)
            try
                N = app.N;
                D = app.D;

                if isempty(N) || isempty(D)
                    error('Ingresa coeficientes numericos en
ambos campos. Ejemplo: 1 2 1');
                end

                validateattributes(N, {'numeric'}, {'row', 'real'
, 'finite'}, '', 'Numerador');
                validateattributes(D, {'numeric'}, {'row', 'real'
, 'finite'}, '', 'Denominador');

                if all(D == 0)
                    error('El denominador debe tener al menos un
coeficiente distinto de cero.');
                end

                sis = tf(N, D);

                switch app.v
                    case 'Escalon'
                        [y, t] = step(sis);
                    case 'Impulso'
```

```
                        [y, t] = impulse(sis);
                    otherwise
                        uialert(app.UIFigure, 'Elegi Escalon o
    Impulso.', 'Tipo de entrada');
                        return
                end

                plot(app.UIAxes, t, y, 'LineWidth', 1.5);
                grid(app.UIAxes, 'on');
                xlabel(app.UIAxes, 'Tiempo (s)');
                ylabel(app.UIAxes, 'Salida');
                title(app.UIAxes, 'Respuesta del sistema');

            catch ME
                uialert(app.UIFigure, ME.message, 'Error al
    calcular la respuesta');
            end
        end

```

5. **Prueba**: Ingresemos la siguiente función de transferencia y probamos con una entrada escalón e impulso:

$$G(s) = \frac{1}{s^2 + s + 1} \tag{1}$$

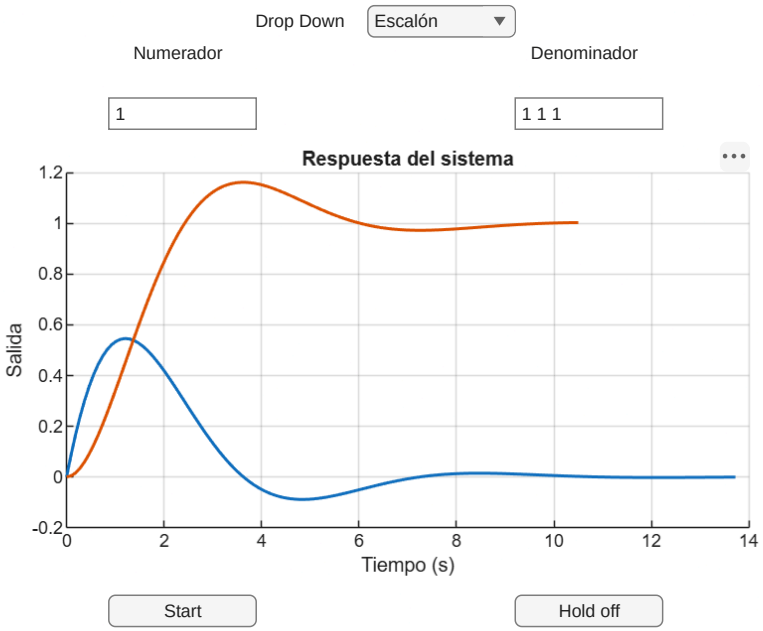


Figura 2. La curva azul representa a la respuesta impulso y la naranja al escalón

El código completo es:

```
classdef app1 < matlab.apps.AppBase

    % Properties that correspond to app components
    properties (Access = public)
        UIFigure          matlab.ui.Figure
        StartButton       matlab.ui.control.Button
        HoldonButton      matlab.ui.control.Button
        DropDown          matlab.ui.control.DropDown
        DropDownLabel     matlab.ui.control.Label
        DenominadorLabel  matlab.ui.control.Label
        numerador         matlab.ui.control.Label
        denEditField      matlab.ui.control.EditField
        numEditField      matlab.ui.control.EditField
        UIAxes            matlab.ui.control.UIAxes
    end


    properties (Access = private)
        N = []
        D = []
        v = 'Escalon'
    end


    % Callbacks that handle component events
    methods (Access = private)

        % Value changed function: numEditField
        function numEditFieldValueChanged(app, event)
            app.N = str2num(app.numEditField.Value, Evaluation="
    restricted");
            disp(app.N)
        end

        % Value changed function: denEditField
        function denEditFieldValueChanged(app, event)
            app.D = str2num(app.denEditField.Value, Evaluation="
    restricted");
            disp(app.D)
        end

        % Value changed function: DropDown
        function DropDownValueChanged(app, event)
            app.v = app.DropDown.Value;

        end

        % Button pushed function: HoldonButton
        function HoldonButtonPushed(app, event)
            if ishold(app.UIAxes)
                hold(app.UIAxes, 'off');
                app.HoldonButton.Text = 'Hold on';
            else
                hold(app.UIAxes, 'on');
                app.HoldonButton.Text = 'Hold off';
            end
        end

        % Button pushed function: StartButton
```

```
58          function StartButtonPushed(app, event)
59              try
60                  N = app.N;
61                  D = app.D;
62
63                  if isempty(N) || isempty(D)
64                      error('Ingresa coeficientes numericos en ambos
    campos. Ejemplo: 1 2 1');
65                  end
66
67                  validateattributes(N, {'numeric'}, {'row', 'real', '
    finite'}, '', 'Numerador');
68                  validateattributes(D, {'numeric'}, {'row', 'real', '
    finite'}, '', 'Denominador');
69
70                  if all(D == 0)
71                      error('El denominador debe tener al menos un
    coeficiente distinto de cero.');
72                  end
73
74                  sis = tf(N, D);
75
76                  switch app.v
77                      case 'Escalon'
78                          [y, t] = step(sis);
79                      case 'Impulso'
80                          [y, t] = impulse(sis);
81                      otherwise
82                          uialert(app.UIFigure, 'Elegi Escalon o Impulso
    .', 'Tipo de entrada');
83                          return
84                  end
85
86                  plot(app.UIAxes, t, y, 'LineWidth', 1.5);
87                  grid(app.UIAxes, 'on');
88                  xlabel(app.UIAxes, 'Tiempo (s)');
89                  ylabel(app.UIAxes, 'Salida');
90                  title(app.UIAxes, 'Respuesta del sistema');
91
92              catch ME
93                  uialert(app.UIFigure, ME.message, 'Error al calcular
    la respuesta');
94              end
95          end
96      end
97
98      % Component initialization
99      methods (Access = private)
100
101         % Create UIFigure and components
102         function createComponents(app)
103
104             % Create UIFigure and hide until all components are
    created
105             app.UIFigure = uifigure('Visible', 'off');
106             app.UIFigure.Color = [1 1 1];
107             app.UIFigure.Position = [100 100 640 480];
108             app.UIFigure.Name = 'MATLAB App';
109             app.UIFigure.Theme = 'light';
110             app.UIFigure.WindowStyle = 'alwaysontop';
111             app.UIFigure.WindowState = 'minimized';
```

```
112
113             % Create UIAxes
114             app.UIAxes = uiaxes(app.UIFigure);
115             title(app.UIAxes, 'Title')
116             xlabel(app.UIAxes, 'X')
117             ylabel(app.UIAxes, 'Y')
118             zlabel(app.UIAxes, 'Z')
119             app.UIAxes.Position = [68 66 506 290];
120
121             % Create numEditField
122             app.numEditField = uieditfield(app.UIFigure, 'text');
123             app.numEditField.ValueChangedFcn = createCallbackFcn(app,
    @numEditFieldValueChanged, true);
124             app.numEditField.Position = [137 365 100 22];
125
126             % Create denEditField
127             app.denEditField = uieditfield(app.UIFigure, 'text');
128             app.denEditField.ValueChangedFcn = createCallbackFcn(app,
    @denEditFieldValueChanged, true);
129             app.denEditField.Position = [410 365 100 22];
130
131             % Create numerador
132             app.numerador = uilabel(app.UIFigure);
133             app.numerador.Position = [154 406 65 22];
134             app.numerador.Text = 'Numerador';
135
136             % Create DenominadorLabel
137             app.DenominadorLabel = uilabel(app.UIFigure);
138             app.DenominadorLabel.Position = [421 406 77 22];
139             app.DenominadorLabel.Text = 'Denominador';
140
141             % Create DropDownLabel
142             app.DropDownLabel = uilabel(app.UIFigure);
143             app.DropDownLabel.HorizontalAlignment = 'right';
144             app.DropDownLabel.Position = [231 427 65 22];
145             app.DropDownLabel.Text = 'Drop Down';
146
147             % Create DropDown
148             app.DropDown = uidropdown(app.UIFigure);
149             app.DropDown.Items = {'Entrada', 'Escalon', 'Impulso'};
150             app.DropDown.ValueChangedFcn = createCallbackFcn(app,
    @DropDownValueChanged, true);
151             app.DropDown.Position = [311 427 100 22];
152             app.DropDown.Value = 'Escalon';
153
154             % Create HoldonButton
155             app.HoldonButton = uibutton(app.UIFigure, 'push');
156             app.HoldonButton.ButtonPushedFcn = createCallbackFcn(app,
    @HoldonButtonPushed, true);
157             app.HoldonButton.Position = [410 31 100 22];
158             app.HoldonButton.Text = 'Hold on';
159
160             % Create StartButton
161             app.StartButton = uibutton(app.UIFigure, 'push');
162             app.StartButton.ButtonPushedFcn = createCallbackFcn(app,
    @StartButtonPushed, true);
163             app.StartButton.Position = [137 31 100 22];
164             app.StartButton.Text = 'Start';
165
166             % Show the figure after all components are created
167             app.UIFigure.Visible = 'on';
```

```
168         end
169     end
170 
171     % App creation and deletion
172     methods (Access = public)
173 
174         % Construct app
175         function app = app1
176 
177             % Create UIFigure and components
178             createComponents(app)
179 
180             % Register the app with App Designer
181             registerApp(app, app.UIFigure)
182 
183             if nargout == 0
184                 clear app
185             end
186         end
187 
188         % Code that executes before app deletion
189         function delete(app)
190 
191             % Delete UIFigure when app is deleted
192             delete(app.UIFigure)
193         end
194     end
195 end
```

Guía 24: Especialidades en MATLAB

El siguiente código de MATLAB representa un generador de números aleatorios donde ninguno de ellos se repite:

```
clear all
close all
clc

a = randi([1, 15], 1, 15);
S = [];

while true
    for i = a
        if ismember(i, S)
            n = length(S);
            a = randi([1, 15], 1, 15 - n);
            break;
        end
        S = [S, i];
    end
    m = length(S);
    if m == 15
        break;
    end
end

S
>> 15  12  10  8  4  5  9  7  3  13  14  1  2  11  6
```

Básicamente lo que se hizo es crear un arreglo de números aleatorios, es decir, la variable `a`. Fue necesario crear otra variable, `S`, donde guardaremos todos los números aleatorios que no se repiten. Al ponerlo dentro de un bucle *while* éste proceso se repetirá hasta que `S` no tenga ningun elemento repetido. `ismember` es una función que nos dice si dicho elemento está en `S` o no. En el caso que esté, se creará un nuevo arreglo pero en lugar de 15 será un arreglo de 15 - `n`, donde `n` es el tamaño de `S`. Cabe aclarar que este código se hizo sin ayuda de IA.

Existe una función interna de MATLAB que hace lo mismo y es `randperm`:

```
r = randperm(15)
>> 4  10  5  14  13  9  15  2  6  8  12  11  7  1  3
```

En el anterior código se escribió lo siguiente:

```
S = [];
.
.
.
S = [S, i];
```

La variable `S` no está definida en tamaño, por lo tanto su tamaño irá cambiando a medida que le agreguemos elementos. MATLAB advierte lo siguiente:

> Variable appears to change size on very loop iteration. Consider preallocating for speed.

Básicamente hay que definir desde un principio el tamaño del arreglo. Veamos los siguientes 2 ejemplos

```
tic
v = [];
i = 1;
```

```
while i<sqrt(7777777)
 v = [ v i^2 ];
 i = i + 1;
end
toc
>> Elapsed time is 0.008991 seconds.


tic
n = floor(sqrt(7777777));
v = nan(1, n);
i = 1;
while i<sqrt(7777777)
 v(i) = i^2;
 i = i + 1;
end
toc
>> Elapsed time is 0.005196 seconds.
```

En el primer caso definimos un arreglo vacío y en el segundo predefinimos el tamaño que tendrá la lista. Se observa que el segundo ejemplo fue 57.79 % más rápido que el primer ejemplo. Por lo tanto, definir el tamaño del arreglo resulta una ventaja en el tiempo de compilación.